In [1]:
```
# Install OpenCV
!pip install opencv-python matplotlib numpy Pillow

# Import necessary libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import os
from google.colab import files
import io
import math

print("Libraries installed and imported successfully!")
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-pa
ckages (5.0.0.93)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packa
ges (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages
(2.1.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.13/dist-packages
(11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist
-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-pac
kages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dis
t-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dis
t-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-
packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist
-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/
dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-package
s (from python-dateutil>=2.7->matplotlib) (1.17.0)
Libraries installed and imported successfully!
```

In [2]:
```
def upload_image():
    uploaded = files.upload()
    if not uploaded:
        print("No file uploaded.")
        return None, None

    for filename in uploaded.keys():
        print(f'User uploaded file "{filename}" with length {len(uploaded[filena
        image_bytes = uploaded[filename]
        return filename, image_bytes
    return None, None

def display_image(title, img_cv, cmap=None):
    if img_cv is None:
        print(f"Cannot display image for '{title}' as it is None.")
        return

    plt.figure(figsize=(10, 8))
```

```
    # OpenCV loads images as BGR, Matplotlib expects RGB. Convert if necessary.
    if len(img_cv.shape) == 3 and img_cv.shape[2] == 3:
        plt.imshow(cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB), cmap=cmap)
    else:
        plt.imshow(img_cv, cmap=cmap)
    plt.title(title)
    plt.axis('off')
    plt.show()

def display_images_side_by_side(title1, img_cv1, title2, img_cv2, cmap1=None, cm
    plt.figure(figsize=(15, 7))

    plt.subplot(1, 2, 1)
    if len(img_cv1.shape) == 3 and img_cv1.shape[2] == 3:
        plt.imshow(cv2.cvtColor(img_cv1, cv2.COLOR_BGR2RGB), cmap=cmap1)
    else:
        plt.imshow(img_cv1, cmap=cmap1)
    plt.title(title1)
    plt.axis('off')

    plt.subplot(1, 2, 2)
    if len(img_cv2.shape) == 3 and img_cv2.shape[2] == 3:
        plt.imshow(cv2.cvtColor(img_cv2, cv2.COLOR_BGR2RGB), cmap=cmap2)
    else:
        plt.imshow(img_cv2, cmap=cmap2)
    plt.title(title2)
    plt.axis('off')

    plt.tight_layout()
    plt.show()
```

In [3]:

```
def view_image_properties():
    print("**Task 1: Image Properties Viewer**")
    print("Please upload a JPG, PNG, or BMP image.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    # Check file extension
    ext = os.path.splitext(filename)[1].lower()
    if ext not in ['.jpg', '.jpeg', '.png', '.bmp']:
        print(f"Error: Unsupported file format '{ext}'. Please upload JPG, PNG,
        return

    try:
        # Read image using OpenCV from bytes
        np_array = np.frombuffer(image_bytes, np.uint8)
        img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

        if img is None:
            raise ValueError("Could not decode image. It might be corrupted or a

        height, width = img.shape[:2]
        channels = 1 if len(img.shape) == 2 else img.shape[2]
        data_type = img.dtype
        file_size_bytes = len(image_bytes)
        file_size_mb = file_size_bytes / (1024 * 1024)
```

```
        print("\n--- Image Properties ---")
        print(f"Filename: {filename}")
        print(f"Dimensions: {width}x{height} pixels")
        print(f"Number of Channels: {channels}")
        print(f"Data Type: {data_type}")
        print(f"File Size: {file_size_bytes} bytes ({file_size_mb:.2f} MB)")
        print("-------------------------")

        display_image(f"Uploaded Image: {filename}", img)

    except Exception as e:
        print(f"An error occurred: {e}")

view_image_properties()
```

```
**Task 1: Image Properties Viewer**
Please upload a JPG, PNG, or BMP image.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2025-11-14 234933.png to Screenshot 2025-11-14 234933.png
User uploaded file "Screenshot 2025-11-14 234933.png" with length 139043 bytes

--- Image Properties ---
Filename: Screenshot 2025-11-14 234933.png
Dimensions: 456x511 pixels
Number of Channels: 4
Data Type: uint8
File Size: 139043 bytes (0.13 MB)
-------------------------
```

Uploaded Image: Screenshot 2025-11-14 234933.png



Task 2: Colour to Grayscale Conversion

This program reads a colour image, converts it to grayscale, displays both original and grayscale images side by side, and saves both versions.

In [4]:

```
def convert_to_grayscale():
    print("**Task 2: Colour to Grayscale Conversion**")
    print("Please upload a colour image.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
        original_img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)
```

```
        if original_img is None:
            raise ValueError("Could not decode image.")

        if len(original_img.shape) == 2 or original_img.shape[2] == 1: # Already
            print("Warning: Uploaded image is already grayscale or 1-channel. Sk
            grayscale_img = original_img
        else:
            grayscale_img = cv2.cvtColor(original_img, cv2.COLOR_BGR2GRAY)

        # Display images
        display_images_side_by_side(
            f"Original: {filename}", original_img,
            f"Grayscale: {filename}", grayscale_img, cmap2='gray'
        )

        # Save images
        original_output_path = f"original_{filename}"
        grayscale_output_path = f"grayscale_{filename}"

        cv2.imwrite(original_output_path, original_img)
        cv2.imwrite(grayscale_output_path, grayscale_img)

        print(f"Original image saved as '{original_output_path}'")
        print(f"Grayscale image saved as '{grayscale_output_path}'")

        # Offer to download
        files.download(original_output_path)
        files.download(grayscale_output_path)

    except Exception as e:
        print(f"An error occurred: {e}")

convert_to_grayscale()
```

```
**Task 2: Colour to Grayscale Conversion**
Please upload a colour image.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2025-12-02 110906.png to Screenshot 2025-12-02 110906.png
User uploaded file "Screenshot 2025-12-02 110906.png" with length 282991 bytes
```

Original: Screenshot 2025-12-02 110906.png



Grayscale: Screenshot 2025-12-02 110906.png



```
Original image saved as 'original_Screenshot 2025-12-02 110906.png'
Grayscale image saved as 'grayscale_Screenshot 2025-12-02 110906.png'
```

Task 3: Image Format Converter

This program takes one input image, converts and saves it as JPG, PNG, and BMP, calculates and compares the file sizes of all three formats, and displays the comparison clearly.

In [5]:

```python
def convert_image_formats():
    print("**Task 3: Image Format Converter**")
    print("Please upload an image for format conversion.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
        img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

        if img is None:
            raise ValueError("Could not decode image.")

        base_name = os.path.splitext(filename)[0]
        formats = {'jpg': '.jpg', 'png': '.png', 'bmp': '.bmp'}
        file_sizes = {}

        print("\n--- Converting and Saving Formats ---")
        output_paths = []
        for fmt_name, ext in formats.items():
            output_filename = f"{base_name}_converted{ext}"
            cv2.imwrite(output_filename, img)
            file_size = os.path.getsize(output_filename)
```

```
            file_sizes[fmt_name] = file_size
            output_paths.append(output_filename)
            print(f"Saved as {output_filename} (Size: {file_size / 1024:.2f} KB)")
        print("-------------------------------------")

        print("\n--- File Size Comparison ---")
        # Sort formats by size for clearer comparison
        sorted_sizes = sorted(file_sizes.items(), key=lambda item: item[1])
        for fmt, size in sorted_sizes:
            print(f"- {fmt.upper()}: {size / 1024:.2f} KB")
        print("----------------------------")

        # Display the original image
        display_image(f"Original Image: {filename}", img)

        # Offer to download all converted files
        for path in output_paths:
            files.download(path)

    except Exception as e:
        print(f"An error occurred: {e}")

convert_image_formats()
```

```
**Task 3: Image Format Converter**
Please upload an image for format conversion.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2025-09-12 190507.png to Screenshot 2025-09-12 190507.png
User uploaded file "Screenshot 2025-09-12 190507.png" with length 205447 bytes

--- Converting and Saving Formats ---
Saved as Screenshot 2025-09-12 190507_converted.jpg (Size: 50.61 KB)
Saved as Screenshot 2025-09-12 190507_converted.png (Size: 268.30 KB)
Saved as Screenshot 2025-09-12 190507_converted.bmp (Size: 775.30 KB)
-------------------------------------

--- File Size Comparison ---
- JPG: 50.61 KB
- PNG: 268.30 KB
- BMP: 775.30 KB
----------------------------
```

Original Image: Screenshot 2025-09-12 190507.png



Task 4: Photo Negative Generator

This program accepts an uploaded image, generates its negative/complement using image processing, displays the original and negative images side by side, and saves the negative image.

In [6]:
```
def generate_photo_negative():
    print("**Task 4: Photo Negative Generator**")
    print("Please upload an image to generate its negative.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
```

```
        original_img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

        if original_img is None:
            raise ValueError("Could not decode image.")

        # Generate negative image: For an 8-bit image, negative = 255 - pixel_va
        negative_img = 255 - original_img

        # Display images
        display_images_side_by_side(
            f"Original: {filename}", original_img,
            f"Photo Negative: {filename}", negative_img
        )

        # Save the negative image
        base_name = os.path.splitext(filename)[0]
        ext = os.path.splitext(filename)[1]
        negative_output_path = f"{base_name}_negative{ext}"

        cv2.imwrite(negative_output_path, negative_img)
        print(f"Negative image saved as '{negative_output_path}'")

        # Offer to download
        files.download(negative_output_path)

    except Exception as e:
        print(f"An error occurred: {e}")

generate_photo_negative()
```

```
**Task 4: Photo Negative Generator**
Please upload an image to generate its negative.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2025-12-02 112546.png to Screenshot 2025-12-02 112546.png
User uploaded file "Screenshot 2025-12-02 112546.png" with length 264407 bytes
```

Original: Screenshot 2025-12-02 112546.png

Photo Negative: Screenshot 2025-12-02 112546.png



```
Negative image saved as 'Screenshot 2025-12-02 112546_negative.png'
```

Task 5: Batch Image Resizer

This program reads multiple images, resizes them while preserving their original aspect ratio, saves the resized copies to an output folder, and allows the user to specify the maximum width or height.

In [7]:
```
def batch_image_resizer():
    print("**Task 5: Batch Image Resizer**")
    print("Please upload multiple images to resize.")

    uploaded_files = files.upload()
    if not uploaded_files:
        print("No files uploaded.")
        return

    # User input for max dimension
    max_dim_input = input("Enter maximum width or height (e.g., '500' for 500 pi
    try:
        max_dim = int(max_dim_input)
        if max_dim <= 0:
            raise ValueError("Max dimension must be a positive integer.")
    except ValueError as e:
        print(f"Invalid input: {e}. Aborting resize operation.")
        return

    output_folder = "resized_images"
    os.makedirs(output_folder, exist_ok=True)
    print(f"Resized images will be saved in: {output_folder}/")

    resized_image_paths = []

    for filename, image_bytes in uploaded_files.items():
        try:
            np_array = np.frombuffer(image_bytes, np.uint8)
            img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

            if img is None:
                print(f"Error: Could not decode image {filename}. Skipping.")
                continue

            h, w = img.shape[:2]
            scale = 1.0

            if w > h:
                if w > max_dim:
                    scale = max_dim / w
            else:
                if h > max_dim:
                    scale = max_dim / h

            new_w = int(w * scale)
            new_h = int(h * scale)

            if new_w == 0 or new_h == 0: # Avoid zero dimensions
                print(f"Warning: Calculated new dimensions for {filename} are ze
                continue
```

```
            resized_img = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTE

            output_path = os.path.join(output_folder, f"resized_{filename}")
            cv2.imwrite(output_path, resized_img)
            resized_image_paths.append(output_path)
            print(f"Resized {filename} from {w}x{h} to {new_w}x{new_h} and saved

            # Display original and resized images for each processed image
            display_images_side_by_side(
                f"Original: {filename}", img,
                f"Resized ({new_w}x{new_h}): {filename}", resized_img
            )

        except Exception as e:
            print(f"An error occurred while processing {filename}: {e}")

    print("\n--- Batch Resizing Complete ---")
    if resized_image_paths:
        print("You can now download the resized images from the 'resized_images'
        # Optional: Offer to download all resized images as a zip
        # import shutil
        # shutil.make_archive('resized_images_archive', 'zip', output_folder)
        # files.download('resized_images_archive.zip')
    else:
        print("No images were successfully resized.")

batch_image_resizer()
```

```
**Task 5: Batch Image Resizer**
Please upload multiple images to resize.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2026-02-26 202627.png to Screenshot 2026-02-26 202627.png
Enter maximum width or height (e.g., '500' for 500 pixels): 500
Resized images will be saved in: resized_images/
Resized Screenshot 2026-02-26 202627.png from 345x433 to 345x433 and saved as res
ized_images/resized_Screenshot 2026-02-26 202627.png
```

Original: Screenshot 2026-02-26 202627.png



Resized (345x433): Screenshot 2026-02-26 202627.png



```
--- Batch Resizing Complete ---
You can now download the resized images from the 'resized_images' folder.
```

Task 6: Region of Interest (ROI) Cropper

This program loads an image, allows the user to specify ROI coordinates (`x` , `y` , `width` , `height`), crops the selected region, displays the original and cropped images, and saves the cropped region as a separate file.

In [8]:
```
def crop_roi():
    print("**Task 6: Region of Interest (ROI) Cropper**")
    print("Please upload an image to crop.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
        original_img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

        if original_img is None:
            raise ValueError("Could not decode image.")

        h, w = original_img.shape[:2]
        print(f"Image dimensions: {w}x{h}")
        print("Please specify the ROI coordinates (x, y, width, height).")
        print("Coordinates are 0-indexed, from top-left corner.")

        x_str = input("Enter x-coordinate (column start): ")
        y_str = input("Enter y-coordinate (row start): ")
        width_str = input("Enter width: ")
        height_str = input("Enter height: ")

        x, y, crop_w, crop_h = int(x_str), int(y_str), int(width_str), int(heigh

        # Validate ROI coordinates
        if not (0 <= x < w and 0 <= y < h and x + crop_w <= w and y + crop_h <=
            print("Error: Invalid ROI coordinates. Ensure x,y are within bounds
            return

        # Crop the image
        cropped_img = original_img[y:y+crop_h, x:x+crop_w]

        # Display images
        display_images_side_by_side(
            f"Original: {filename}", original_img,
            f"Cropped ROI ({x},{y},{crop_w},{crop_h}): {filename}", cropped_img
        )

        # Save the cropped image
        base_name = os.path.splitext(filename)[0]
        ext = os.path.splitext(filename)[1]
        cropped_output_path = f"{base_name}_cropped_x{x}_y{y}_w{crop_w}_h{crop_h

        cv2.imwrite(cropped_output_path, cropped_img)
        print(f"Cropped image saved as '{cropped_output_path}'")
```

```
        # Offer to download
        files.download(cropped_output_path)

    except ValueError:
        print("Error: Invalid input. Please enter integer values for coordinates
    except Exception as e:
        print(f"An error occurred: {e}")

crop_roi()
```

```
**Task 6: Region of Interest (ROI) Cropper**
Please upload an image to crop.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving hacathon2.jpg to hacathon2.jpg
User uploaded file "hacathon2.jpg" with length 109952 bytes
Image dimensions: 960x1280
Please specify the ROI coordinates (x, y, width, height).
Coordinates are 0-indexed, from top-left corner.
Enter x-coordinate (column start): 32
Enter y-coordinate (row start): 36
Enter width: 56
Enter height: 60
```

Original: hacathon2.jpg



Cropped ROI (32,36,56,60): hacathon2.jpg



```
Cropped image saved as 'hacathon2_cropped_x32_y36_w56_h60.jpg'
```

In [9]:

```
def rgb_channel_split_reconstruction():
    print("**Task 7: RGB Channel Split and Reconstruction**")
    print("Please upload a color image (JPG, PNG).")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
        original_bgr = cv2.imdecode(np_array, cv2.IMREAD_COLOR) # Ensure color i

        if original_bgr is None:
            raise ValueError("Could not decode image or it is not a color image.
```

```
        # Split channels (OpenCV default BGR order)
        b, g, r = cv2.split(original_bgr)

        print("\n--- Displaying Individual Channels ---")
        # Display each channel (grayscale representation of intensity)
        display_image(f"Blue Channel ({filename})", b, cmap='gray')
        display_image(f"Green Channel ({filename})", g, cmap='gray')
        display_image(f"Red Channel ({filename})", r, cmap='gray')

        # Reconstruct the image from channels (order: B, G, R)
        reconstructed_bgr = cv2.merge([b, g, r])

        # Display original and reconstructed for visual comparison
        display_images_side_by_side(
            f"Original Image: {filename}", original_bgr,
            f"Reconstructed Image: {filename}", reconstructed_bgr
        )

        # Verify pixel-level accuracy
        if np.array_equal(original_bgr, reconstructed_bgr):
            print("\nReconstruction is PERFECTLY accurate at the pixel level!\n"
        else:
            print("\nReconstruction is NOT perfectly accurate at the pixel level

    except Exception as e:
        print(f"An error occurred: {e}")

rgb_channel_split_reconstruction()
```

```
**Task 7: RGB Channel Split and Reconstruction**
Please upload a color image (JPG, PNG).
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving FB_IMG_1656652839360.jpg to FB_IMG_1656652839360.jpg
User uploaded file "FB_IMG_1656652839360.jpg" with length 175821 bytes

--- Displaying Individual Channels ---
```

Blue Channel (FB_IMG_1656652839360.jpg)



Green Channel (FB_IMG_1656652839360.jpg)








```
Reconstruction is PERFECTLY accurate at the pixel level!
```

In [11]:

```python
def image_watermarking_utility():
    print("**Task 8: Image Watermarking Utility**")
    print("Please upload the main image to watermark.")

    filename, image_bytes = upload_image()
    if filename is None:
        return

    try:
        np_array = np.frombuffer(image_bytes, np.uint8)
        original_img = cv2.imdecode(np_array, cv2.IMREAD_UNCHANGED)

        if original_img is None:
            raise ValueError("Could not decode main image.")
```

```
    # Ensure original_img is 3-channel for consistent watermarking
    if len(original_img.shape) == 2:
        original_img = cv2.cvtColor(original_img, cv2.COLOR_GRAY2BGR)
    elif original_img.shape[2] == 4: # RGBA to BGR
        original_img = cv2.cvtColor(original_img, cv2.COLOR_BGRA2BGR)

    watermarked_img = original_img.copy()
    h, w, _ = watermarked_img.shape

    watermark_type = input("Enter watermark type ('text' or 'logo'): ").lowe
    if watermark_type not in ['text', 'logo']:
        print("Invalid watermark type. Please enter 'text' or 'logo'.")
        return

    alpha_input = input("Enter transparency (0.0 to 1.0, e.g., 0.5 for 50% t
    try:
        alpha = float(alpha_input)
        if not (0.0 <= alpha <= 1.0):
            raise ValueError("Transparency must be between 0.0 and 1.0.")
    except ValueError as e:
        print(f"Invalid transparency value: {e}. Defaulting to 0.5.")
        alpha = 0.5

    # Position input
    print("\nEnter watermark position (e.g., 'bottom-right', 'top-left', 'ce
    print("For custom, enter 'custom' then x, y coordinates.")
    pos_input = input("Position: ").lower()

    pos_x, pos_y = 0, 0 # Default to top-left for calculation
    custom_pos = False
    if pos_input == 'custom':
        custom_pos = True
        try:
            pos_x = int(input("Enter custom X coordinate: "))
            pos_y = int(input("Enter custom Y coordinate: "))
        except ValueError:
            print("Invalid coordinates. Using default top-left.")
            pos_x, pos_y = 10, 30 # Default for text

    if watermark_type == 'text':
        text = input("Enter watermark text: ")
        font = cv2.FONT_HERSHEY_SIMPLEX
        font_scale = 1.5 # Increased font scale for better visibility
        font_thickness = 3 # Increased thickness
        text_color = (0, 0, 0) # Black color for text

        # Calculate text size to determine position
        text_size = cv2.getTextSize(text, font, font_scale, font_thickness)[
        text_w, text_h = text_size[0], text_size[1]

        if not custom_pos:
            if pos_input == 'bottom-right':
                pos_x = w - text_w - 20 # 20 pixel padding
                pos_y = h - text_h - 20 # 20 pixel padding
            elif pos_input == 'top-left':
                pos_x = 20
                pos_y = text_h + 20 # Add padding and text height
            elif pos_input == 'center':
                pos_x = (w - text_w) // 2
```

```
                pos_y = (h + text_h) // 2
            else: # Default to bottom-left if unrecognized
                pos_x = 20
                pos_y = h - 20

        # Ensure text position is within bounds
        pos_x = max(0, min(pos_x, w - text_w))
        pos_y = max(text_h, min(pos_y, h - 10)) # Ensure Y is not below 0 an

        # Create a transparent overlay for text
        overlay = watermarked_img.copy()
        cv2.putText(overlay, text, (pos_x, pos_y), font, font_scale, text_co
        watermarked_img = cv2.addWeighted(overlay, alpha, watermarked_img, 1

    elif watermark_type == 'logo':
        print("Please upload the logo image for watermarking.")
        logo_filename, logo_bytes = upload_image()
        if logo_filename is None:
            print("No logo uploaded. Aborting watermarking.")
            return

        logo = cv2.imdecode(np.frombuffer(logo_bytes, np.uint8), cv2.IMREAD_
        if logo is None:
            raise ValueError("Could not decode logo image.")

        # Resize logo if too large (e.g., to 1/5th of the main image's width
        logo_h, logo_w = logo.shape[:2]
        if logo_w > w / 3 or logo_h > h / 3: # If logo is larger than 1/3 of
            scale = min((w/3) / logo_w, (h/3) / logo_h)
            logo = cv2.resize(logo, (int(logo_w * scale), int(logo_h * scale
            logo_h, logo_w = logo.shape[:2]

        # Handle logo channels (convert to 3-channel BGR if needed)
        if len(logo.shape) == 2:
            logo = cv2.cvtColor(logo, cv2.COLOR_GRAY2BGR)
        elif logo.shape[2] == 4: # RGBA logo to BGR for overlaying
            b, g, r, a = cv2.split(logo)
            logo = cv2.merge([b, g, r]) # Use the BGR channels
            # Use alpha channel as mask for blending
            logo_alpha_mask = a / 255.0
            # Convert alpha mask to 3 channels for consistent blending
            logo_alpha_mask = cv2.merge([logo_alpha_mask, logo_alpha_mask, l
        else:
            logo_alpha_mask = np.ones((logo_h, logo_w, 3), dtype=np.float32)

        if not custom_pos:
            if pos_input == 'bottom-right':
                pos_x = w - logo_w - 20
                pos_y = h - logo_h - 20
            elif pos_input == 'top-left':
                pos_x = 20
                pos_y = 20
            elif pos_input == 'center':
                pos_x = (w - logo_w) // 2
                pos_y = (h - logo_h) // 2
            else: # Default to bottom-right if unrecognized
                pos_x = w - logo_w - 20
                pos_y = h - logo_h - 20

        # Ensure logo position is within bounds
```

```
            pos_x = max(0, min(pos_x, w - logo_w))
            pos_y = max(0, min(pos_y, h - logo_h))

            # Create ROI for placing the logo
            roi = watermarked_img[pos_y : pos_y + logo_h, pos_x : pos_x + logo_w

            # Blend logo into ROI with transparency (using alpha for overlay and
            # Calculate final alpha for blending
            final_alpha_logo = alpha * logo_alpha_mask # Apply user-defined alph

            watermarked_img[pos_y : pos_y + logo_h, pos_x : pos_x + logo_w] = \
                (roi * (1 - final_alpha_logo) + logo * final_alpha_logo).astype(

        # Display images
        display_images_side_by_side(
            f"Original: {filename}", original_img,
            f"Watermarked: {filename}", watermarked_img
        )

        # Save the watermarked image
        base_name = os.path.splitext(filename)[0]
        ext = os.path.splitext(filename)[1]
        watermarked_output_path = f"{base_name}_watermarked{ext}"

        cv2.imwrite(watermarked_output_path, watermarked_img)
        print(f"Watermarked image saved as '{watermarked_output_path}'")

        # Offer to download
        files.download(watermarked_output_path)

    except ValueError as ve:
        print(f"Input Error: {ve}")
    except Exception as e:
        print(f"An error occurred: {e}")

image_watermarking_utility()
```

```
**Task 8: Image Watermarking Utility**
Please upload the main image to watermark.
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

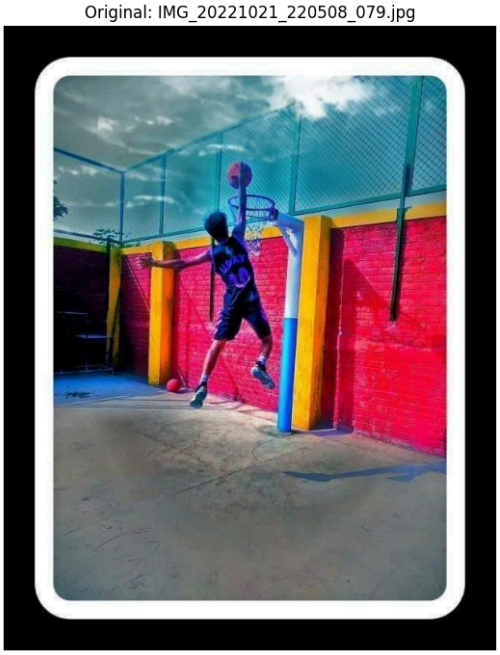
```
Saving IMG_20221021_220508_079.jpg to IMG_20221021_220508_079.jpg
User uploaded file "IMG_20221021_220508_079.jpg" with length 72220 bytes
Enter watermark type ('text' or 'logo'): logo
Enter transparency (0.0 to 1.0, e.g., 0.5 for 50% transparent): 60%
Invalid transparency value: could not convert string to float: '60%'. Defaulting
to 0.5.

Enter watermark position (e.g., 'bottom-right', 'top-left', 'center').
For custom, enter 'custom' then x, y coordinates.
Position: 35,56
Please upload the logo image for watermarking.
```

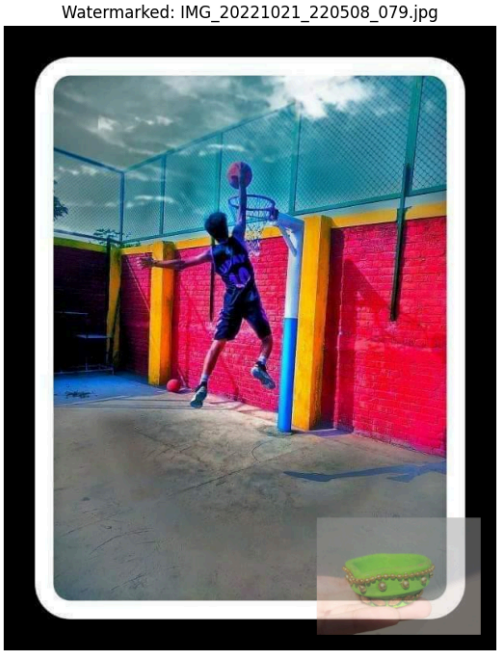
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving IMG_20231023_104017_844.jpg to IMG_20231023_104017_844.jpg
User uploaded file "IMG_20231023_104017_844.jpg" with length 44969 bytes
```

Original: IMG_20221021_220508_079.jpg



Watermarked: IMG_20221021_220508_079.jpg



```
Watermarked image saved as 'IMG_20221021_220508_079_watermarked.jpg'
```
